

Great—this is exactly the **right upgrade** if your goal is *non-trivial*, context-aware speech control.

Below is a **minimal but correct LLM-agent architecture** for **Unity/Godot**, where:

- Speech → text
- **LLM understands intent + context**
- **LLM chooses tools (functions)**, not keywords
- Game logic remains authoritative

This avoids turning your game into a chatbot while still benefiting from LLM reasoning.

1. Core Principle (Very Important)

LLM = planner / interpreter

Game = executor / authority

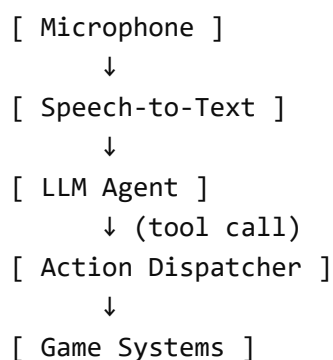
The LLM:

- Interprets player intent
- Selects a tool (function)
- Provides arguments

The game:

- Validates
 - Executes
 - Returns state updates
-

2. High-Level Architecture (LLM Agent Loop)





</> Code



This feedback loop is what gives **context awareness**.

3. Minimal Agent Contract (Engine-Agnostic)

Tool (Function) Definitions

These are the **only things** the LLM can do.

```
[
  {
    "name": "move_to",
    "description": "Move the player to a location",
    "parameters": {
      "type": "object",
      "properties": {
        "location": { "type": "string" }
      },
      "required": ["location"]
    }
  },
  {
    "name": "water_crops",
    "description": "Water crops in an area",
    "parameters": {
      "type": "object",
      "properties": {
        "area": { "type": "string" }
      }
    }
  },
  {
    "name": "harvest",
    "description": "Harvest mature crops",
    "parameters": {
      "type": "object",
      "properties": {
        "target": { "type": "string" }
      }
    }
  }
]
```

4. World State (Context Injection)

Before every LLM call, provide **structured state**, not prose.

</> JSON



```
{
  "player": {
    "location": "barn",
    "holding": "watering_can"
  },
  "nearby_objects": [
    { "type": "crop", "status": "dry", "area": "north_field" },
    { "type": "chicken", "count": 3 }
  ],
  "last_action": "move_to(barn)"
}
```

</> JSON



5. System Prompt (Critical)

This is what makes the LLM behave like a **game agent**, not a chatbot.

```
You are a game control agent.
You do not explain your reasoning.
You do not talk to the player.
You ONLY respond by calling one of the provided tools.
If no tool applies, respond with "no_action".
Prefer the most specific valid action.
```

</> Plain text



6. Unity Implementation (Minimal, Correct)

LLMClient.cs (pseudo-code)

```
public async Task<LLMResponse> Interpret(string text, WorldState state)
{
    var request = new {
        messages = new [] {
            new { role="system", content=SYSTEM_PROMPT },
            new { role="user", content=text },
        }
    };
}
```

```

        new { role="user", content=JsonUtility.ToJson(state) }
    },
    tools = ToolDefinitions
};

return await CallLLM(request);
}

```

</> C#



Action Dispatcher

```

void Execute(LLMResponse response)
{
    if (!response.HasToolCall) return;

    switch (response.ToolName)
    {
        case "move_to":
            player.MoveTo(response.Args["location"]);
            break;

        case "water_crops":
            farm.Water(response.Args["area"]);
            break;
    }
}

```

</> C#



7. Code Implementation (Game Concepts)

```

func on_llm_response(resp):
    match resp.tool:
        "move_to":
            player.move_to(resp.args.location)
        "water_crops":
            farm.water(resp.args.area)

```

</> gdscript



8. Handling Ambiguity (Non-Trivial Control)

If LLM confidence is low or multiple targets exist:

```
{
  "name": "request_clarification",
  "parameters": {
    "question": "Which field should I water?"
  }
}
```

↔ JSON



Then,

- Game asks player (voice or UI)
- Response becomes next context turn

9. Why This Is Better Than Keywords

Keywords	LLM Agent
Fragile phrasing	Robust paraphrases
No memory	Contextual memory
Hard-coded logic	Semantic reasoning
Breaks immersion	Natural speech

10. Common Mistakes to Avoid

- ✗ Letting the LLM move objects directly
 - ✗ Passing raw scene graphs
 - ✗ Allowing free-text responses
 - ✗ Treating LLM output as authoritative
 - ✓ Tools + validation + feedback loop
-

11. Minimal Tool Set (Start Small)

Start with 5–7 tools max:

- move_to
- interact

- use_item
- water
- harvest
- repeat_last_action

Expand only after user testing.

12. Next Steps (I can help you implement)

If you want, I can:

- Provide a **Unity-ready OpenAI / local LLM wrapper**
- Design a **farming-specific tool schema**
- Show **state compression** (important for latency)
- Add **logging for research evaluation**

Tell me:

1. Unity or Godot?
2. Local LLM or API?
3. Target latency (real-time vs turn-based)?

This is a strong, research-grade setup—you're on the right track.

